Assignment_2_Data_Wrangling_Visualization

2026-09

```
# Loading the packages I'll use in this assignment
library(tidyverse) # Our primary data wrangling set of packages
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(gridExtra) # A package that helps arranging ggplot objects
```

```
##
## Attaching package: 'gridExtra'
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(ggpubr) # For the statistical tests - Wilcoxon rank sum test
```

1) Generating random data is a useful skill. An exercise such as this is not only useful for developing data wrangling skills, but we can also use the resulting data for training and testing sets in machine learning. For this first exercise, I would like you to generate a data frame of random data with the following attributes:

a) The data frame should contain the following columns:

- Participant_ID: Generate reasonable participant IDs such as "Participant_001", or something similar.
- Sex: Male and Female
- Age: Should follow a normal distribution and be bound between the ages 18 and 90.
- Activity_Status: Should contain the levels "Inactive", "Lightly Active", "Average Activity", and "Very Active".
- Disease_Status: Should contain "Case" and "Control" levels. Participants with a systolic blood pressure greater than or equal to 150 and a diastolic blood pressure greater than or equal to 95 should be considered a "Case", otherwise the participant should be considered a "Control".

- Systolic_Blood_Pressure: The distribution from which the systolic blood pressure values should be taken is a normal distribution with a minimum of 110 and a maximum of 200. The mean should be 120 and the standard deviation should be 30.
- Diastolic_Blood_Pressure: The diastolic blood pressures should be dependent on the systolic blood pressures if a systolic blood pressure is less than or equal to 120 than the diastolic blood pressure should follow a normal distribution where the minimum is 70 and the maximum is 100 with a mean of 80 and a standard deviation of 5. Otherwise, the diastolic blood pressure should have a minimum of 90 and a maximum of 150. This should also follow a normal distribution. The mean should be 100 abd the standard deviation should be 5.

b) The data set should contain 100 participants

```r
## Plan
# 1. I'll set my seed for the sampling, then I'll set a number of participants to sample
 ↪  as a variable
# 2. I'll build a dataframe with all the variables that aren't dependent on others. I'll
 ↪  use rnorm and sample for the sampling as appropriate.
# 3. I'll mutate a variable categorizng the systolic blood pressure to above and below
 ↪  120, then group by that variable so I can apply the different
#    Diastolic blood pressure sampling distributions based on what group the participant
 ↪  falls into.
# 4. Once I've grouped I'll create a diastolic blood pressure variable, sampling based on
 ↪  group status as the instructions say to
# 5. Then I'll ungroup the data, and use the systolic value and the derived diastolic
 ↪  value to add the disease status value.

# Set a seed for sample reproducibility
set.seed(123)

# The number of participants to sample
n <- 100

# Generate random data
random_data <- data.frame(
  Participant_ID = paste("pt", 1:n, sep = "_"),  # Generate Subject IDs
  Sex = sample(c("Male", "Female"), n, replace = TRUE),  # Random sex values
  Age = pmax(18, pmin(90, round(rnorm(n, mean = 50, sd = 10)))),  # Random ages between
   ↪  18 and 90
  Activity_Status = sample(c("Inactive", "Lightly Active", "Average Activity", "Very
   ↪  Active"), n, replace = TRUE),  # Random activity status values
  Systolic_Blood_Pressure = pmax(110, pmin(200, round(rnorm(n, mean = 120, sd = 30))))  #
   ↪  Random systolic blood pressure values
) %>%
  mutate(systolic_group = case_when(
    Systolic_Blood_Pressure <= 120 ~ "less_120", # Create the grouping variable based on
 ↪  systolic BP
    Systolic_Blood_Pressure > 120 ~ "gr_120",
    TRUE ~ NA
  )) %>%
  group_by(systolic_group) %>% # Group by the new variable
  # Execute the specific sampling pattern depending on the group the participant falls
   ↪  into
  mutate(
    Diastolic_Blood_Pressure = if (cur_group()$systolic_group == "less_120") {
```

```
      pmax(70, pmin(100, round(rnorm(n(), mean = 80, sd = 5))))
    } else {
      pmax(90, pmin(150, round(rnorm(n(), mean = 100, sd = 5))))
    } # #The diastolic blood pressures should be dependent on the systolic blood
↪  pressures if a systolic blood pressure is less than or equal to 120 than the
↪  diastolic blood pressure should follow a normal distribution where the minimum is 70
↪  and the maximum is 100 with a mean of 80 and a standard deviation of 5. Otherwise,
↪  the diastolic blood pressure should have a minimum of 90 and a maximum of 150. This
↪  should also follow a normal distribution. The mean should be 100 abd the standard
↪  deviation should be 5.
  ) %>%
  ungroup() %>%
  mutate(
    Disease_Status = case_when( ## create the disease status variable based on BP values
      Systolic_Blood_Pressure >= 150 & Diastolic_Blood_Pressure >= 95 ~ "Case",
      TRUE ~ "Control"
    ) # Participants with a systolic blood pressure greater than or equal to 150 and a
↪  diastolic blood pressure greater than or equal to 95 should be considered a "Case",
↪  otherwise the participant should be considered a "Control".
  ) %>%
  select(-systolic_group) # remove the grouping variable since it wasn't asked for in the
   ↪  final dataset and is no longer needed

# View the first few rows of the data
head(random_data)
```

```
## # A tibble: 6 x 7
##   Participant_ID Sex      Age Activity_Status Systolic_Blood_Pressure
##   <chr>          <chr>  <dbl> <chr>                             <dbl>
## 1 pt_1           Male      53 Lightly Active                      186
## 2 pt_2           Male      50 Lightly Active                      159
## 3 pt_3           Male      50 Inactive                            112
## 4 pt_4           Female    64 Lightly Active                      136
## 5 pt_5           Male      48 Inactive                            110
## 6 pt_6           Female    65 Inactive                            110
## # i 2 more variables: Diastolic_Blood_Pressure <dbl>, Disease_Status <chr>
```

2) Convert the data frame you generated above from wide to long format based on the systolic and diastolic blood pressure columns. Make sure you take a look at the column names and how the data frame is now formatted after converting to long form!

```
## Plan
# 1. I'll use the tidyvere pivot_longer() function to pivot on the two BP variables

long_random_data <- random_data %>%
  pivot_longer(
    cols = c(Systolic_Blood_Pressure, Diastolic_Blood_Pressure), # The 2 variables you
     ↪  want to pivot
    names_to = "blood_pressure_type", # A new name for the columns
    values_to = "blood_pressure_reading" # A new name for the values
  )

# View the first few rows of the data
```

```r
head(long_random_data)
```

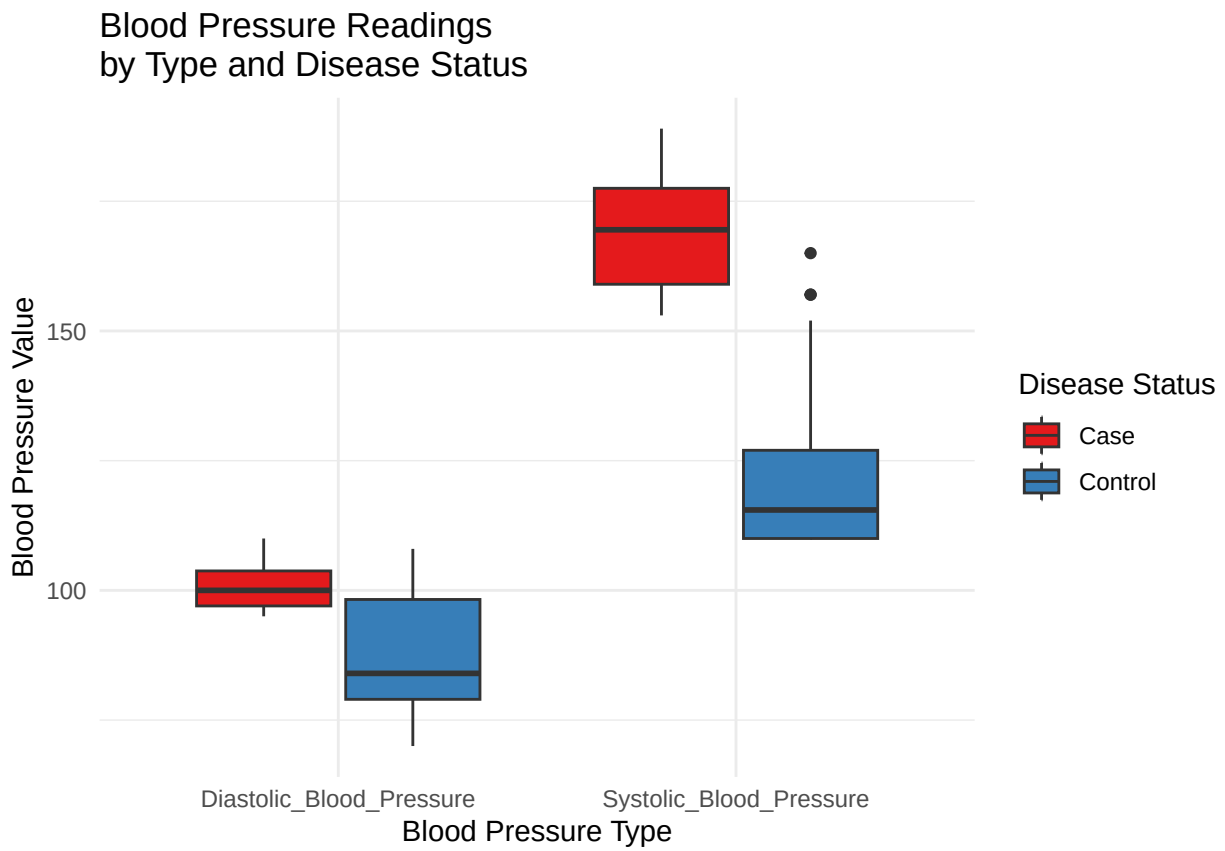
```
## # A tibble: 6 x 7
##   Participant_ID Sex     Age Activity_Status Disease_Status blood_pressure_type
##   <chr>          <chr> <dbl> <chr>           <chr>          <chr>
## 1 pt_1           Male     53 Lightly Active  Case           Systolic_Blood_Pres~
## 2 pt_1           Male     53 Lightly Active  Case           Diastolic_Blood_Pre~
## 3 pt_2           Male     50 Lightly Active  Case           Systolic_Blood_Pres~
## 4 pt_2           Male     50 Lightly Active  Case           Diastolic_Blood_Pre~
## 5 pt_3           Male     50 Inactive        Control        Systolic_Blood_Pres~
## 6 pt_3           Male     50 Inactive        Control        Diastolic_Blood_Pre~
## # i 1 more variable: blood_pressure_reading <dbl>
```

3) As Data Scientists, we will be asked to generate some neat looking figures. In the following questions, generate ggplot figures using the long format data you generated in the previous question, unless otherwise specified.

a) Generate a grouped ggplot2 box plot of the systolic and diastolic blood pressures in cases and controls. The x-axis should be the blood pressure types. The y-axis should be the blood pressure values. The fill should be the disease status: case or control.

```r
## Plan
# 1. use the ggplot function with geom_boxplot() to build the boxplot with the prescribed
 ↪  x, y and fill using the long data
# 2. add clear labels and title
# 3. use theme_minimal() to create a clean white background
# 4. Add the Set1 color scheme for box plot fills

ggplot(long_random_data, aes(x = blood_pressure_type, y = blood_pressure_reading, fill =
 ↪  Disease_Status)) + # Set variables to use
  geom_boxplot() + # Add type of plot
  labs(x = "Blood Pressure Type", y = "Blood Pressure Value", fill = "Disease Status",
       title = "Blood Pressure Readings \nby Type and Disease Status") + # Add labels and
        ↪  title
  theme_minimal() + # Create white background
  scale_fill_brewer(palette = "Set1") # Add color scheme for fills
```

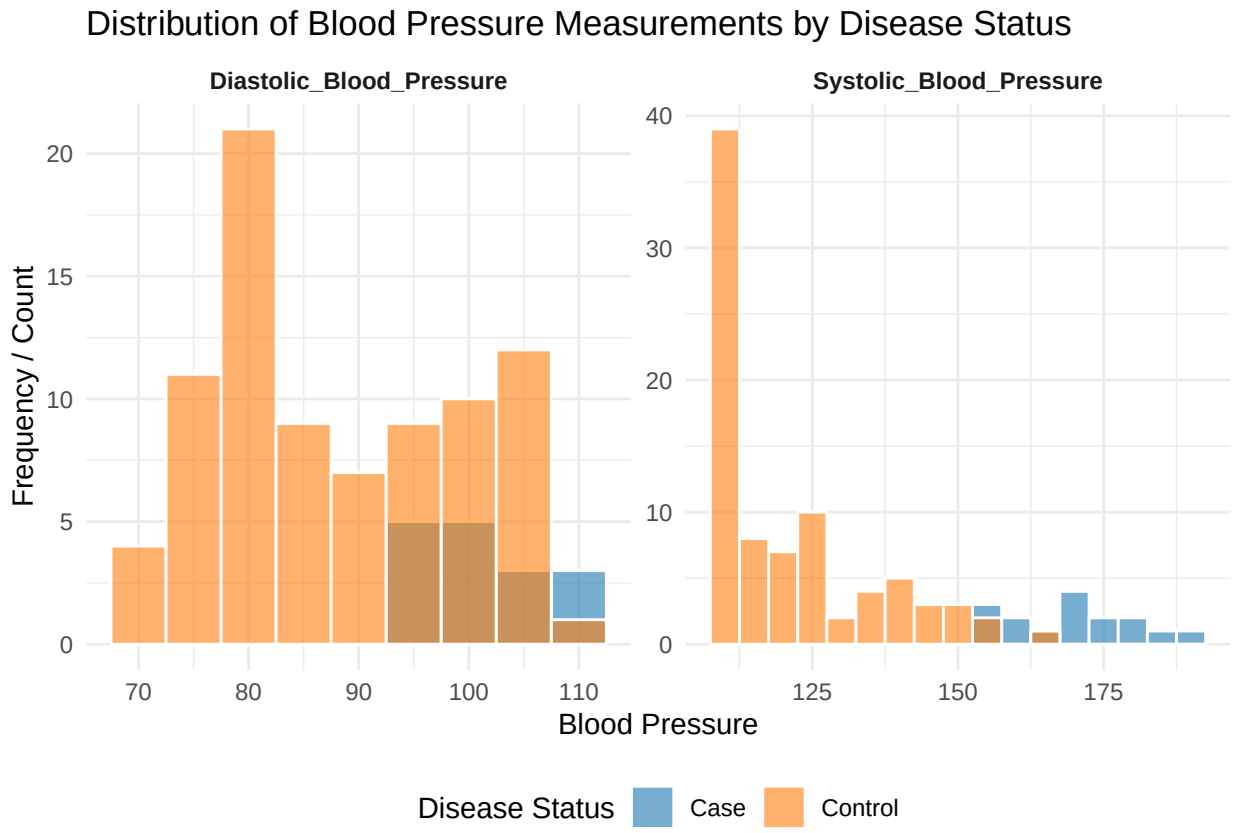


b) Generate a ggplot2 histogram of the systolic and diastolic blood pressures. Color the bars by disease status: case and control.

```
## Plan
# 1. use the ggplot function with geom_histogram() to build the histogram using the long
 ↪  data
# 2. Set the x axis as blood pressure readings, the y axis will be count of frequency of
 ↪  observations with that reading
# 3. Set the width to 5 so it is easier to read the chart
# 4. Set other histogram parameters to standards better for visualizing
# 5. add facet layer so there are two graphs for systolic and diastolic
# 6. Add colors for fills of disease status
# 7. use theme_minimal() to create a clean white background
# 8. add some more formatting
# 4. Add labeling and titling

ggplot(long_random_data, aes(x = blood_pressure_reading, fill = Disease_Status)) +
  # Histogram parameters
  geom_histogram(
    binwidth = 5,           # Each bin contains a range of 5 different integer readings
    alpha = 0.6,            # Makes overlap in bars see-through
    position = "identity",  # Keeps bars independent but on top of each other (which is
     ↪  why the overlap being see-through matters)
    color = "white"         # Add a border separating bars next to each other so it's
     ↪  easier to see
```

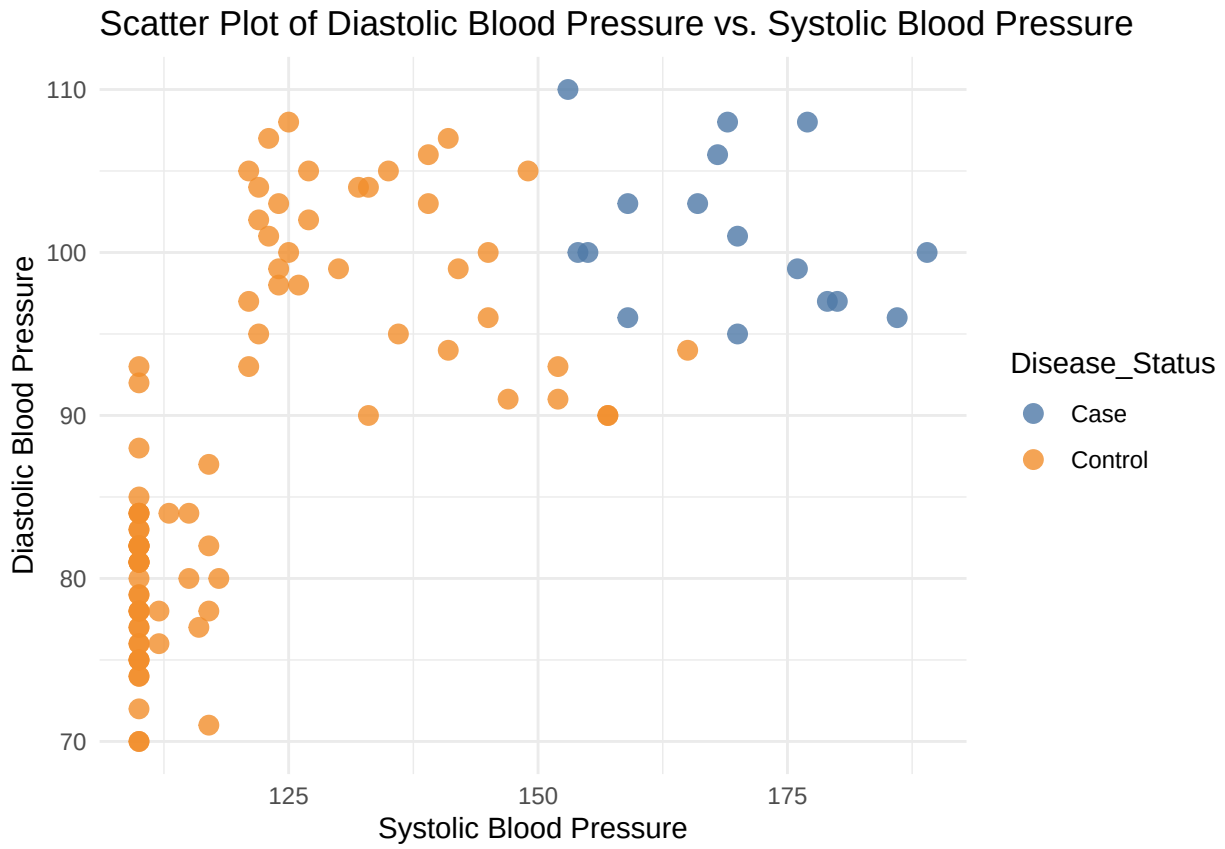
```r
) +
# Facet layer (split the plot into two panels for Systolic and Diastolic)
# scales = "free" allows the axes to adjust to fit each test type value range
facet_wrap(~ blood_pressure_type, scales = "free", ncol = 2) +
# Color Customizing
scale_fill_manual(
  values = c(
    "Case"      = "#1f77b4", # Blue
    "Control"   = "#ff7f0e"  # Orange
  )
) +
theme_minimal() +
theme(
  legend.position = "bottom",          # put the disease status legend at the bottom
  strip.text = element_text(face="bold") # bold "Systolic" and "Diastolic" labels
) +
# Labels
labs(
  title = "Distribution of Blood Pressure Measurements by Disease Status",
  x = "Blood Pressure",
  y = "Frequency / Count",
  fill = "Disease Status"
)
```



c) Using the wide format data frame, generate a scatter plot of systolic blood pressure on the x-axis and diastolic blood pressure on the y-axis. Once again, color the points by disease status: case and control.

```r
## Plan
# 1. use the ggplot function with geom_point() to build the histogram using the wide data
# 2. Plot the two blood pressure types on the x and y axis as requested
# 3. Add a layer to add color to the points based on disease status
# 4. use theme_minimal() for a white background

ggplot(random_data, aes(x = Systolic_Blood_Pressure, y = Diastolic_Blood_Pressure)) + # Define x and y
  # creates the scatter plot and add the color aesthetic
  geom_point(aes(color = Disease_Status), size = 3, alpha = 0.8) + # alpha refers to the opacity
  # Add colors based on disease status
  scale_color_manual(
    values = c(
      "Case" = "#4e79a7", # Blue
      "Control" = "#f28e2b"  # Orange
    )
  ) +
  theme_minimal() +
  # Labels
  labs(
    title = "Scatter Plot of Diastolic Blood Pressure vs. Systolic Blood Pressure",
    x = "Systolic Blood Pressure",
    y = "Diastolic Blood Pressure"
  )
```



4) Building off the code you used to generate the box plot above, add code that runs a wilcoxon rank sum test and display the resulting p-values comparing the cases and controls in the diastolic and systolic blood pressures on your box plot. Save your figure as a .png file and upload it with the submission for this assignment. After completing this, report the summary statistics for the systolic and diastolic blood pressures separately.

```
## Plan
# 1. Define the png name for my boxplot output
# 2. Add the same boxplot code as above, but add a stat_compare_means() function to pull
 ↪  the requested p values
# 3. set the method to wilcox.test() for the wilcoxon rank sum test and explicitly define
 ↪  the grouping variable
# 4. run dev.off() to output the boxplot in the png
# 5. Add other wilcoxon test tables as a gut check the ggplot is pulling the correct p
 ↪  values for the right comparison
# 6. Generate summary stats tables of the two BP variables

png("bp_boxplot.png", width = 3000, height = 2000, res = 300) # set png name and width
 ↪  and height so all is read correctly

ggplot(long_random_data, aes(x = blood_pressure_type, y = blood_pressure_reading, fill =
 ↪  Disease_Status)) +
  geom_boxplot() +
  labs(x = "Blood Pressure Type", y = "Blood Pressure Value", fill = "Disease Status",
       title = "Blood Pressure Readings \nby Type and Disease Status") +
  theme_minimal() +
  scale_fill_brewer(palette = "Set1") +
  stat_compare_means(aes(group = Disease_Status), # Add wilcoxon test and define grouping
   ↪  variable
    method = "wilcox.test"
  )

dev.off() # output png
```

```
## pdf
##   2
```

```
wilcoxon_summary_stats_sys <- wilcox.test( ## Gut Check of systolic BP p value
  blood_pressure_reading ~ Disease_Status,
  data = subset(long_random_data, blood_pressure_type == "Systolic_Blood_Pressure")
)

print(wilcoxon_summary_stats_sys) # Output
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  blood_pressure_reading by Disease_Status
## W = 1333, p-value = 1.838e-10
## alternative hypothesis: true location shift is not equal to 0
```

```r
wilcoxon_summary_stats_diast <- wilcox.test( ## Gut Check of diastolic BP p value
  blood_pressure_reading ~ Disease_Status,
  data = subset(long_random_data, blood_pressure_type == "Diastolic_Blood_Pressure")
)

print(wilcoxon_summary_stats_diast) # Output
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  blood_pressure_reading by Disease_Status
## W = 1107.5, p-value = 4.269e-05
## alternative hypothesis: true location shift is not equal to 0
```

```r
systolic_summary_stats <- long_random_data %>% # Summary stats for systolic BP using
 ↪  summarise()
  filter(blood_pressure_type == "Systolic_Blood_Pressure") %>%
  summarise("Number of Participants" = n_distinct(Participant_ID), # number of
   ↪  participants vs. readings
            "Count Observations" = n(),
            "Mean Systolic Blood Pressure" = mean(blood_pressure_reading),
            "Median Systolic Blood Pressure" = median(blood_pressure_reading),
            "Max Systolic Blood Pressure" = max(blood_pressure_reading),
            "Min Systolic Blood Pressure" = min(blood_pressure_reading),
            "IQR Systolic Blood Pressure" = IQR(blood_pressure_reading) # diff between
             ↪  25th and 75th percentile
  )

print(systolic_summary_stats) # output
```

```
## # A tibble: 1 x 7
##   `Number of Participants` `Count Observations` `Mean Systolic Blood Pressure`
##                      <int>                <int>                          <dbl>
## 1                      100                  100                           129.
## # i 4 more variables: `Median Systolic Blood Pressure` <dbl>,
## #   `Max Systolic Blood Pressure` <dbl>, `Min Systolic Blood Pressure` <dbl>,
## #   `IQR Systolic Blood Pressure` <dbl>
```

```r
diastolic_summary_stats <- long_random_data %>% # Summary stats for diastolic BP using
 ↪  summarise()
  filter(blood_pressure_type == "Diastolic_Blood_Pressure") %>%
  summarise("Number of Participants" = n_distinct(Participant_ID), # number of
   ↪  participants vs. readings
            "Count Observations" = n(),
            "Mean Diastolic Blood Pressure" = mean(blood_pressure_reading),
            "Median Diastolic Blood Pressure" = median(blood_pressure_reading),
            "Max Diastolic Blood Pressure" = max(blood_pressure_reading),
            "Min Diastolic Blood Pressure" = min(blood_pressure_reading),
            "IQR Diastolic Blood Pressure" = IQR(blood_pressure_reading) # diff between
             ↪  25th and 75th percentile
  )

print(diastolic_summary_stats) # output
```

```
## # A tibble: 1 x 7
##   `Number of Participants` `Count Observations` `Mean Diastolic Blood Pressure`
##                      <int>                <int>                            <dbl>
## 1                      100                  100                             90.2
## # i 4 more variables: `Median Diastolic Blood Pressure` <dbl>,
## #   `Max Diastolic Blood Pressure` <dbl>, `Min Diastolic Blood Pressure` <dbl>,
## #   `IQR Diastolic Blood Pressure` <dbl>
```

5) Remove the systolic and diastolic blood pressure columns in the wide format data frame. Generate a heart rate column instead and add that to the data frame. The heart rates should be dependent on the level of activity of the participant. Participants who are "Inactive" should have a heart rate taken from a normal distribution with a minimum of 100 and a maximum of 200. The mean of this distribution should be 150 and have a standard deviation of 20. If the participant is "Lightly Active", their heart rate should be taken from a normal distribution with a minimum of 80 and a maximum of 99. The mean of this distribution should be 85 and the standard deviation should be 20. Finally, if the participant is "Average Activity" or "Very Active," their heart rate should be taken from a normal distribution with a minimum of 40 and a maximum of 70. The mean of this distribution should be 50 and the standard deviation 20.

```r
## Plan
# 1. create a new data frame, set it to the original wide data
# 2. use the pipe operator to manipulate it, removing the BP columns
# 3. group by activity status and mutate based on a conditional that evaluates based on
 ↪  group status to the specific sampling requested
# 4. show the first few rows of the data

random_data_heart_rate <- random_data %>%
  select(-Systolic_Blood_Pressure, -Diastolic_Blood_Pressure) %>% # remove bp columns
  group_by(Activity_Status) %>% # group by activity status
  # Run the specific sampling pattern depending on the activity status group
  mutate(
    Heart_Rate = if (cur_group()$Activity_Status == "Inactive") { # use a conditional to
     ↪  determine which sampling pattern
      pmax(100, pmin(200, round(rnorm(n(), mean = 150, sd = 20))))
    } else if (cur_group()$Activity_Status == "Lightly Active") {
      pmax(80, pmin(99, round(rnorm(n(), mean = 85, sd = 20))))
    } else {
      pmax(40, pmin(70, round(rnorm(n(), mean = 50, sd = 20))))
    }
  ) %>%
  ungroup()

head(random_data_heart_rate) # show first few rows of the data
```

```
## # A tibble: 6 x 6
##   Participant_ID Sex      Age Activity_Status Disease_Status Heart_Rate
##   <chr>          <chr>  <dbl> <chr>           <chr>                <dbl>
## 1 pt_1           Male      53 Lightly Active  Case                    80
## 2 pt_2           Male      50 Lightly Active  Case                    90
## 3 pt_3           Male      50 Inactive        Control                186
## 4 pt_4           Female    64 Lightly Active  Control                 80
## 5 pt_5           Male      48 Inactive        Control                156
## 6 pt_6           Female    65 Inactive        Control                153
```